

TP — O termostato

Tudo o que a bancada construiu, num programa só, apresentado e defendido

Microcontroladores — DENE/UFMT

OBJETIVOS

- Integrar numa aplicação as peças dos roteiros: medida, display, base de tempo, interrupção, PWM, controle, serial e memória.
- Tomar decisões de projeto que não têm resposta única, e justificá-las com números medidos ou calculados.
- Organizar o código de forma que outra pessoa encontre cada coisa, e que uma mudança não quebre o resto.
- Demonstrar o sistema funcionando e responder, individualmente, por qualquer parte dele.

1 A aplicação

Um termostato que mantém a temperatura do conjunto LM35 e resistor num **alvo ajustável**, usando **dois atuadores por PWM**: o aquecedor, para subir, e o cooler, para descer. O alvo se ajusta por botões, aparece no display junto com a temperatura e as duas saídas, sobrevive a desligar a placa, e tudo o que acontece sai pela serial para um arquivo.

Nenhuma peça é nova. Cada requisito abaixo já foi feito, isolado, em algum roteiro. O trabalho do TP é o que nenhum roteiro fez: fazer as peças conviverem no mesmo programa — dividindo pinos, temporizadores, tempo de processador e memória — sem que uma estrague a outra.

CONCEITO

Chave	Posição	NESTE KIT
CH1-7 (TEMP)	ON — LM35 em RA0/AN0	
CH1-1, CH1-5 e CH1-6	OFF — também chegam a RA0	
CH2-1 (LCD)	ON	
CH3-3 (HEATER, RC1 = CCP2)	ON	
CH3-5 (COOLER, RC2 = CCP1)	ON	
CH3-1 (tacômetro, RC0)	ON na Parte 1; opcional na Parte 2	
CH4-6 e CH4-8 (RS232)	ON — serial em RC6/RC7	
CH4-5 e CH4-7 (RS485) e CH5 (relés)	OFF — os relés disputam RC6/RC7 e PORTD	

A verificar no seu kit, antes de escrever código: que o CCP2 sai em RC1 (bit de configuração CCP2MX, gravado com o bootloader) e que o aquecedor responde a PWM na frequência que o grupo escolher.

2 Duas partes

Parte	O que é	Roteiros	Apresentação
1 — o painel	O termostato mede, mostra, conta e decide liga-desliga, sem interrupção e sem PWM	R4 a R6	semana 7
2 — o termostato	A mesma base, agora com interrupção, PWM nos dois atuadores, botões, serial e memória	R4 a R11	encontro 15

A Parte 2 é construída **sobre** a Parte 1: os módulos que o grupo escrever agora são os que vão ser reaproveitados no fim. Código bem organizado na Parte 1 é tempo ganho na Parte 2.

3 Parte 1 — o painel

Com o que existe até o R6 — display, conversor e temporizadores —, o sistema já pode medir, mostrar e decidir. Ainda sem interrupção: tudo acontece no laço, e por isso o laço não pode esperar.

3.1 Requisitos da Parte 1

	Requisito	Origem
A1	Medir a temperatura em décimos de grau, sem ponto flutuante, a 10 amostras por segundo	R5
A2	Mostrar no display a temperatura, a rotação do cooler em rpm e um relógio mm:ss desde a partida, com largura fixa e sem tremular	R4
A3	Base de tempo por temporizador, sem <code>__delay_ms</code> no laço. O relógio pode atrasar no máximo 2 s em 10 min de cronômetro	R6
A4	Rotação do cooler: o Timer1 conta o tacômetro em RC0 durante janelas de 1 s, convertida para rpm	R6
A5	Liga-desliga com um alvo fixo em <code>config.h</code> : abaixo de alvo – 1 °C liga o aquecedor, acima de alvo + 1 °C liga o cooler, entre os dois nenhum	—

O relógio é a parte difícil. O R6 mostrou que recarregar o Timer0 na consulta faz o intervalo derivar, e o laço desta parte escreve no display. Com um tique de 10 ms e uma tela de 1,3 ms, a conta ingênua dá segundos de atraso em dez minutos. O grupo precisa medir o seu atraso contra um cronômetro e, se passar do limite, resolver — a folha de referência dos temporizadores tem mais de uma saída.

ATENÇÃO

3.2 Decisões da Parte 1

Decisão	O que pesa
Como o relógio evita a deriva	Onde acontece a recarga, quanto o laço demora no pior caso, e se existe um temporizador que se recarrega sozinho
Pré-carga e divisor	O intervalo, o passo e o maior valor que cabe em 16 bits (R6)
Janela da rotação	Um pulso a mais ou a menos numa janela de 1 s vale quantos rpm?
Atualização do display	A tela custa 1,3 ms; a temperatura muda em minutos. Quantas vezes por segundo vale escrever?
Aritmética	O maior produto da conversão para décimos, e o tipo que o comporta (R5)

3.3 Organização da Parte 1

Os arquivos `config.h`, `main.c`, `adc.c/.h`, `lcd.c/.h`, `tempo.c/.h` e `tacometro.c/.h`, com as regras 1, 5 e 6 da Parte 2: laço sem espera, maior produto comentado, e uma linha no topo de cada arquivo dizendo o que ele faz.

3.4 Entregar e apresentar a Parte 1

Na véspera da semana 7: o projeto compactado com `LEIAME.md`, a folha de decisões (meia página, com número em cada linha) e a medida do relógio contra o cronômetro em 10 minutos.

Na semana 7, dez minutos por grupo: três de demonstração — o display, o cooler ligado com a rotação na tela, e o dedo no sensor fazendo o liga-desliga reagir — e sete de arguição, **com cada integrante respondendo pelo menos uma pergunta**. Perguntas típicas: por que `TMR0H` antes de `TMR0L`; quanto o relógio atrasou e por quê; quantos rpm vale um pulso a mais na janela; qual o maior produto da conversão.

3.5 Critério da Parte 1

A Parte 1 vale **15% da nota de laboratório**. Dentro dela:

Item	Pontos
------	--------

Funcionamento — temperatura e display (A1, A2): 1,5; relógio dentro de 2 s em 10 min (A3): 1,0; rotação em rpm (A4): 1,5; liga-desliga (A5): 1,0	5,0
Código — módulos e config.h: 1,0; laço sem espera: 0,6; aritmética verificada: 0,4	2,0
Decisões — folha com número em cada decisão	1,0
Arguição — individual	2,0

4 Parte 2 — o termostato

4.1 Requisitos da Parte 2

Cada requisito indica o roteiro de onde a peça vem. Os **básicos** são o mínimo para o termostato funcionar; os **completos** fecham o produto; os **extras** valem ponto adicional, até o limite da nota.

Requisito	Origem
B1 Medir a temperatura em décimos de grau, sem ponto flutuante, a 10 amostras por segundo	R5
B2 Mostrar no display a temperatura, o alvo e as duas saídas em %, com largura fixa e sem tremular	R4
B3 Base de tempo por interrupção; nenhum <code>__delay_ms</code> no laço principal	R6, R7
B4 Aquecedor em CCP2 e cooler em CCP1, por PWM, na frequência que o grupo escolher e justificar	R8
B5 Controle proporcional: esforço = $K_p \cdot \text{erro}$, saturado; esforço positivo aciona o aquecedor, negativo o cooler, nunca os dois; uma zona morta em torno do alvo	R9, aula 13
B6 Alvo ajustável por dois botões, de 30,0 a 50,0 °C em passos de 0,5 °C, com filtro de repique	R7
C1 Telemetria a 1 Hz pela serial, em texto: <code>t_ms, temp_d, alvo_d, aq_pct, cool_pct</code>	R10
C2 Alvo guardado na memória não volátil e restaurado ao ligar	R11
X1 Vigia de sobretemperatura pelo comparador, que corta o aquecedor mesmo com o laço travado	R9
X2 Termo integral com proteção contra saturação	aula 13
X3 Ganhos ajustados em Python com os dados do R10 e do R13, e confirmados na bancada	R13
X4 Comandos pela serial: mudar o alvo e os ganhos sem regravar	R10

O esforço nunca liga os dois atuadores ao mesmo tempo. Aquecer e resfriar juntos gasta energia para não chegar a lugar nenhum. A zona morta existe para isso: perto do alvo, os dois ficam desligados, e o controle não alterna entre eles a cada amostra — a mesma lição da histerese do R9.

ATENÇÃO

4.2 Decisões que o grupo precisa tomar

Não há resposta certa única para nenhuma delas. Há resposta **justificada**: cada decisão vai para a folha de decisões com o número que a sustenta.

Decisão	O que pesa
Frequência do PWM	CCP1 e CCP2 dividem o Timer2: os dois atuadores têm a mesma frequência (aula 8). A ventoinha apita abaixo de 20 kHz; resolução cai quando a frequência sobe; o aquecedor precisa aceitar a comutação
K_p e zona morta	Erro de regime do proporcional, oscilação e o degrau de 0,49 °C do conversor (aula 9 e aula 13)
Taxa de amostragem e de atualização da tela	A tela custa 1,3 ms; a planta leva minutos. O que precisa ser rápido, e o que não precisa
Quando gravar o alvo na memória	A memória tem número finito de escritas: gravar a cada toque de botão, ou só quando o alvo parar de mudar
Aritmética	O maior produto de cada conta, e o tipo que o comporta (R5)

4.3 Organização mínima do código

O código vai ser lido por outra pessoa na apresentação. Não se pede nenhuma arquitetura sofisticada — pede-se o mínimo que permite achar as coisas.

Arquivo	O que contém
config.h	Pinos, frequências e parâmetros: alvo padrão, K_p , zona morta, limites. Nenhum número mágico fora daqui
main.c	Só a inicialização e o laço principal
tempo.c/.h	A base de tempo por interrupção e a leitura segura do relógio (R7)
adc.c/.h	Leitura do LM35 em décimos de grau (R5)
pwm.c/.h	Iniciar o Timer2 e os dois CCP; escrever cada razão cíclica (R8)
controle.c/.h	O controle: recebe temperatura e alvo, devolve as duas saídas
botoes.c/.h	O filtro de repique e os eventos de “subir” e “descer” (R7)
uart.c/.h	A fila de transmissão e a linha de telemetria (R10)
memoria.c/.h	Ler e gravar o alvo (R11)
lcd.c/.h	O driver do R4
LEIAME.md	Como compilar, quais chaves do kit, e o que cada arquivo faz, em uma página

Regras.

1. O laço principal não espera: nenhum `__delay_ms` fora da inicialização.
2. O tratamento de interrupção é curto — rearmar, contar, sinalizar. Nada de display, divisão ou formatação dentro dele.
3. Toda variável que o tratamento e o laço compartilham é `volatile`, e as de mais de 8 bits são lidas com a proteção do R7.
4. Cada módulo inclui só os `.h` dos outros; nenhum mexe nos registradores de outro periférico.
5. Aritmética inteira, com o maior produto de cada conta escrito num comentário ao lado dela.
6. Cada arquivo começa com uma linha dizendo o que ele faz.

O laço principal, organizado assim, cabe numa tela:

```
#include "config.h"
#include "tempo.h"
#include "adc.h"
#include "pwm.h"
#include "controle.h"
#include "botoes.h"
#include "uart.h"
#include "memoria.h"
#include "lcd.h"

void main(void)
{
    int16_t temp_d = 0, alvo_d;
    saidas_t s = { 0, 0 }; /* razoes do aquecedor e do cooler */

    ADCON1 = 0x0E; /* so AN0 analogico: primeira linha */
    tempo_iniciar(); adc_iniciar(); pwm_iniciar();
    lcd_iniciar();   uart_iniciar(); botoes_iniciar();
    alvo_d = memoria_ler_alvo(); /* o alvo sobrevive a desligar */
}
```

```

for (;;) {                                     /* nenhuma espera aqui dentro */
    if (tempo_passou_100ms()) {
        temp_d = adc_temperatura_decimos();
        s = controle_atualizar(temp_d, alvo_d);
        pwm_aquecedor(s.aquecedor);
        pwm_cooler(s.cooler);
    }
    if (botoes_evento(&alvo_d)) {             /* subir ou descer 0,5 grau */
        memoria_agendar_gravacao(alvo_d);
    }
    if (tempo_passou_500ms()) {
        tela_mostrar(temp_d, alvo_d, s);
    }
    if (tempo_passou_1s()) {
        telemetria_enviar(temp_d, alvo_d, s);
        memoria_gravar_se_agendado();
    }
}
}

```

O esqueleto é uma sugestão, não uma exigência: os nomes podem mudar. O que não muda é a forma — cada coisa acontece no seu ritmo, perguntando ao relógio se chegou a hora, e nenhuma delas prende o laço. É o escalonador cooperativo da aula 7, escrito à mão.

NOTA

4.4 Entregáveis

Na véspera da apresentação, por grupo:

1. O **projeto** do MPLAB X compactado, sem as pastas build e dist, com o LEIAME.md.
2. Um **registro de telemetria** de pelo menos 20 minutos, com pelo menos duas mudanças de alvo feitas pelos botões (por exemplo, $35 \rightarrow 45 \rightarrow 40$ °C).
3. Um **gráfico** desse registro: temperatura e alvo no tempo, e as duas saídas embaixo, no mesmo eixo de tempo.
4. A **folha de decisões**, de uma página: uma tabela com cada decisão, o valor escolhido e a justificativa com número. Por exemplo: “PWM a 20 kHz: acima da audição; 800 passos, 9,6 bits; o aquecedor comuta sem aquecer o transistor.”

4.5 A apresentação

Quinze minutos por grupo, na bancada, no encontro 15.

Demonstração, 5 minutos. O grupo liga o kit e mostra, ao vivo:

- o alvo mudando pelos botões, e o aquecedor ou o cooler reagindo;
- a telemetria chegando ao computador;
- a placa desligada e religada, com o alvo de volta ao que estava;
- o que acontece quando alguém encosta o dedo no sensor.

Arguição, 10 minutos. O professor abre o código e pergunta. **Cada integrante responde pelo menos uma pergunta**, sobre uma parte que ele não escolheu. Perguntas típicas:

- Onde está o tratamento de interrupção, e o que ele faz? Por que não faz mais?
- Qual variável é *volatile*, e por quê? E qual não precisa ser?
- Qual a frequência do PWM, e o que ela custou em resolução?
- Qual o maior produto da conversão para décimos, e em que tipo ele cabe?
- O que acontece com o controle se a temperatura ficar na zona morta?
- Quantas vezes a memória é gravada numa hora de uso? Quanto ela dura assim?
- Aponte na telemetria o erro de regime, e explique de onde ele vem.

4.6 Critério

A Parte 2 vale **15% da nota de laboratório**. Dentro dela:

Item	Pontos
Funcionamento — medição e display (B1, B2): 0,5; PWM nas duas saídas (B4): 0,5; controle no alvo, com erro de regime e oscilação lidos na telemetria (B5): 1,5; botões com filtro (B6): 0,5; telemetria (C1): 0,5; memória (C2): 0,5	4,0
Código — organização em módulos e config.h: 0,8; laço sem espera e tratamento curto: 0,7; volatile, leitura protegida e aritmética verificada: 0,5	2,0
Decisões — folha de decisões com números: 1,2; gráfico e leitura da telemetria: 0,8	2,0
Apresentação e arguição — individual	2,0
Extras X1 a X4 — até o limite de 10	até +1,0

A nota da arguição é individual. O funcionamento, o código e as decisões são do grupo; a resposta sobre eles é de cada um. Quem não souber explicar a parte que o grupo entregou não recebe a nota dela.

ATENÇÃO

5 Cronograma

Encontro	O que acontece
5	Enunciado do TP distribuído, com as duas partes
6	R6: os temporizadores, a última peça da Parte 1
7	Parte 1: apresentação e arguição, na sessão de laboratório
8 a 12	R7 a R11: as peças da Parte 2 chegam uma por semana
13 e 14	Desenvolvimento fora da sessão; o R13 ajusta os ganhos que a Parte 2 pode usar
Véspera do 15	Entrega do projeto, da telemetria, do gráfico e da folha de decisões
15	Parte 2: apresentação e arguição, na bancada